Smart Lab Attendance

Smart Lab Attendance *(student proposal)*

> **Assessment:** ◆ Fit with adjustments · **Category:** GovTech · **Assessed:** 2026-07-06 · **Status:** awaiting instructor approval

Proposal (as submitted): An app for teachers that automatically registers lab-class attendance using indoor-capable mobile technology (something better than GPS indoors). The app knows the module schedule, is installed on students' phones, and notifies the respective instructors of attendance at the end of each session.

Adjustments required for approval:

- **The presence-detection engine must be built, not borrowed.** Recommended technology: **ultrasonic audio beacons** — a speaker in the lab emits coded, near-inaudible chirps, and each student phone detects them with the team's own C++ DSP (FFT / matched filtering over the microphone stream). This is real, module-legal engineering (audio capture + custom C++ signal processing) and makes the real-time high-frequency loop genuinely load-bearing. BLE RSSI scanning may be fused in as a second signal, with custom filtering/smoothing in C++ — but calling an OS "am I near this beacon?" API alone would leave the C++ decorative.
- **A cohort/presence simulator must drive development** (GovTech pool M1 G14): simulated student devices with signal models and arrival/departure behavior — the team cannot demo with 30 physical phones, and seeded scenarios make detection accuracy testable.
- **Attendance decisions computed in the C++ service** — session windows from the schedule, presence-confidence scoring, late/left-early detection — with Firebase used only for record sync and instructor notifications, not as the decision engine.
- **Privacy by design as an explicit requirement:** informed consent in the student app, data minimization (presence-in-room during scheduled sessions only — no location tracking), and a stated retention policy. The

category's RBAC & audit requirement reinforces this: every record and correction is attributable.

References:

- Apple iBeacon — indoor proximity technology (developer docs)
- Android Wi-Fi RTT — indoor ranging API (developer docs)
- Top Hat — classroom platform with automated attendance

Core Tech Requirements

Legend: ✔ Applies directly | ◆ Applies with domain adaptation | ● Optional | ✘ Not needed | » Second trimester

Core Tech Requirement	Smart Lab Attendance
External Database (Firebase)	✔
Authentication »	✔
Analytics »	✔
C++ Performant Service	✔
Real-time C++ Simulation Loop	✔
Application Communication	✔
Continuous Integration	✔
Data-Driven Architecture	✔
Front-end + Local Backend	✔
Custom Tools Development	◆
2D Graphical Output	◆
3D (strong students only)	✘
Networking System	●
RBAC & Audit Logging — roles + immutable action trail (new)	✔

How each requirement applies:

> *Platform fit:* **Native mobile + Web browser on mobile** — the student app must be native (microphone/BLE access; reference phone Pixel 8a/9a); instructors view live class state and reports on mobile web. Native PC is the development vehicle.

Front end: Student app (consent, session status, my-attendance view), instructor view (live class presence timeline, end-of-session summary). **Languages:** C++, GLSL; HTML/JS/CSS (instructor web view via WASM).

Back end: Presence-detection DSP engine, attendance decision engine (session windows, confidence), cohort simulator, audit log, Firebase sync & notifications. **Languages:** C++.
Backend scope: Server-indicated (T2) — a class shares live attendance state; Firebase bridges T1, an authoritative attendance service is the T2 path.

- *Simulation loop:* the detection engine — continuous microphone capture with FFT/matched-filter chirp detection at audio rate (plus optional BLE RSSI filtering), feeding a per-session presence state machine. This is the project's engineering heart.
- *Data-driven:* module schedules, rooms, beacon chirp codes, cohorts and detection thresholds all defined as data; a new module or room is a data change.
- *Custom tool (adapted):* schedule/room/beacon configuration editor — sessions, rooms, chirp code assignments, cohort rosters.
- *External DB:* attendance records and instructor notifications. *Auth:* applies directly — student/instructor/admin roles. *Analytics »:* attendance trends per module/cohort.
- *2D (adapted):* custom-rendered live presence timeline (who arrived when, confidence bands) and attendance charts — not just tables.
- *Networking:* ● BLE scanning as a secondary signal; instructor live view updates via Firebase in T1.
- *RBAC & audit logging:* students see only their own records; instructors see their modules; every automated mark and every manual correction lands in an immutable audit trail — the integrity story of an attendance system.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the GovTech pool in `milestone-goals-pool-per-category.md` (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**
  Tech & dependencies
  - **Tech:** SDL2 shell (PC dev build) and Android app shell sharing one C++ core; a real-time capture loop pulls microphone frames continuously and drives the presence state machine; C++17/20, CMake, Git.
  - **Prerequisites:** None — foundation of this project.
  - **Feeds into:** Every other goal; the detection DSP (proposed goal below) and M1 G4 ingest run inside this loop.
- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**
  Tech & dependencies
  - **Tech:** Schedules, rooms, chirp-code assignments, cohorts and detection thresholds fully described in JSON (nlohmann/json) with hot-reload; nothing institution-specific hardcoded.
  - **Prerequisites:** G1 (loop & hooks).
  - **Feeds into:** M1 G14 (scenario definitions), M2 G2 (editor edits this data), M2 G15 (timetable import fills it).
- **G3 ★ Debugging, Profiling, Stress Test & CI**
  Tech & dependencies
  - **Tech:** ImGui overlays (detection confidence, SNR, callback load), Tracy profiling of the DSP path, Catch2 tests running detection over recorded audio fixtures (deterministic), GitHub Actions Windows+Linux matrix.
  - **Prerequisites:** G1 (loop to instrument).
  - **Feeds into:** M2 G3; the recorded-fixture tests guard every DSP change against accuracy regressions.
- **G4 Real-time Feed / Queue Engine** — ordered, idempotent ingest of presence events into per-session attendance state.

Tech & dependencies

    - **Tech:** Ordered event queue (detected/lost/re-detected per student) with idempotent processing — a re-sent event never double-marks; running per-session attendance state derived from the stream.
    - **Prerequisites:** G1 (loop), G2 (session/cohort model).
    - **Feeds into:** the end-of-session summary & instructor notification (absorbed into this goal's definition of done), M2 G7 (every event auditable), the live instructor timeline.
- **G14 Domain Simulation Core** — the cohort/presence simulator (adjustment 2) that makes the system developable and testable.
  Tech & dependencies
    - **Tech:** Seeded simulated cohorts — arrival/departure behavior, signal-strength models, edge cases (late arrival, brief exit, phone in bag) — generating the event stream M1 G4 ingests.
    - **Prerequisites:** G1 (clock), G2 (cohort/scenario data).
    - **Feeds into:** All development and demos; detection-accuracy metrics vs simulated ground truth.
- **G17 Presence & Signal Detection Engine** — the project's engineering heart (adjustment 1), added to the GovTech pool from this proposal.
  Tech & dependencies
    - **Tech:** Real-time DSP over the microphone stream — coded ultrasonic chirp detection via FFT/matched filtering — optionally fused with BLE RSSI filtering (smoothing, hysteresis); presence events with confidence scores, all custom C++.
    - **Prerequisites:** G1 (capture loop), G2 (chirp codes & thresholds as data).
    - **Feeds into:** M1 G4 (its events are what the queue ingests); detection accuracy measured against M1 G14 ground truth.

(Note: the end-of-session instructor notification — formerly pick G12 — is absorbed into **G4**'s definition of done: once per-session state exists, closing a session and dispatching the Firebase summary is a thin slice. The freed slot goes to the detection engine.)

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

  - **Tech:** Tracy on the DSP path, battery- and background-behavior validation on the reference phone (Pixel 8a/9a), ASan on MSVC for the PC dev build; interruption handling (calls, screen-off) proven.
  - **Prerequisites:** All M1 goals.
  - **Feeds into:** A student app that survives a real 2-hour lab session without draining the battery or losing the session.

- **G2 ★ Content Editor — Core** — the schedule/room/beacon configuration editor.
  Tech & dependencies
  - **Tech:** Editor for module sessions, rooms, chirp-code assignments and cohort rosters with JSON round-trip and an undo/redo command stack.
  - **Libraries/Software:** Dear ImGui, nlohmann/json
  - **Prerequisites:** M1 G2 (data model).
  - **Feeds into:** Every configured module; M2 G15 imports feed it, M2 G7 audits its changes.
- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**
  Tech & dependencies
  - **Tech:** CMake presets, clang-format + clang-tidy gates, GitHub Actions producing the PC build, the APK and the instructor WASM view; detection-accuracy regression suite on recorded fixtures.
  - **Prerequisites:** M1 G3.
  - **Feeds into:** Release builds; accuracy must not regress for CI to pass.
- **G7 RBAC & Audit Logging** — the category's purple requirement, and this product's integrity backbone.
  Tech & dependencies
  - **Tech:** Roles (student / instructor / admin) enforced across app, editor and records; append-only audit trail of every automated mark and manual correction with actor + timestamp; audit viewer.
  - **Prerequisites:** M1 G4 (events), Firebase Auth.
  - **Feeds into:** The trust story: show a disputed mark, then show its full history.
- **G11 Reports & Transparency Dashboards** — attendance reports per module, cohort and term.

Tech & dependencies

- **Tech:** Custom-rendered attendance charts (per-module trends, per-student summaries, session heat strips) and exportable reports consistent with the audit trail.
- **Prerequisites:** M1 G4 (records), M2 G7 (audited data).
- **Feeds into:** What instructors and admins actually consume at term end.

- **G15 Data Import / Export & Interop** — the proposal's "app gets the schedule of a module".
  Tech & dependencies
  - **Tech:** Timetable import (CSV/iCal) into the schedule model with validation and clear error reporting; attendance export (CSV) for administrative systems.
  - **Prerequisites:** M1 G2 (schedule model), M2 G2 (editor to review imports).
  - **Feeds into:** Real-world fit — no manual re-typing of the semester timetable.

Track Goals & Team Composition

Recommended composition: 6 CS + 2 Design + 0 Art — utility civic product with a low art surface; 0 artists (Design conditionals mandatory), so the second designer slot is strongly advised (used below).

> **Track goals** — suggested Design/Art picks and TEAM notes for this project; deliverables & quality ladders live in the Design, Art and TEAM pool documents.
>
> - **Design M1:**
>   - D1 ★ Features DD — student check-in side + instructor dashboard blueprint, privacy-by-design mandate
>   - D2 Prototype — arrive → detected → confirmed → instructor view, both roles clickable · prototype in Figma
>   - D5 Form & status ergonomics — presence states, privacy indicators (detection on/off), consent flows
>   - D11 Art POC (conditional — no artist) — calm utility styling, iconography, status color system
> - **Design M2:**
>   - D1 ★ Features Design Iteration & Showcase Readiness
>   - D6 Service polished — session start → detection → attendance log → instructor notification, end to end
>   - D8 Status & notification pass — clarity at every state change (detected / missed / manual override)
>   - D11 Art Implementation Quality (conditional)
> - **UI/UX M1:**
>   - U1 ★ UI Standards & Wireframe Baseline — the D5 presence/privacy conventions are assessed in this review
> - **UI/UX M2:**
>   - U2 ★ UX Validation & Final Polished UI — UX validation + final polished UI
>   - U1 ★ Functional UI & Usability — GovTech screens: student check-in view · instructor live dashboard · session history · settings · help; both roles navigable unaided
> - **Art:** 0 artists — the Design conditionals (M1 D11 Art POC / M2 D11 Art Implementation Quality) cover the art surface.

- **TEAM M1:**
    - TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present and working in the prototype, suitable and correctly triggered for this project.
    - TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add or replace audio without recompiling) and reliable multi-channel playback.
- **TEAM M2:**
    - TA1 ★ Overall Audio Experience — GovTech floor: no BGM; ≥5 cues (check-in confirmed, session start/end, error, override), visual pairs for every cue, frequency-coordinated with G17
    - TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel; advanced where the project calls for it).
    - TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase, prototype demo, team post-mortem (wk14)

Generated by the Project Proposal Assessor skill · grounding: live folder · references verified 2026-07-06 · track goals & unified workbooks retrofitted 2026-07-12.